

第 16 讲：SWE-ContextBench：评测 Coding Agent 是否会复用过去经验

Coding Agent Notes (June 2026)

本讲在系列中的位置：#15 讲单个任务内的 context retrieval；本讲讲跨任务的 context learning，也就是过去解决过的问题能否帮助未来相关问题。

已知前提：读者已经知道 context budget、repo intelligence、gold context、recall / precision / F1。

本讲新增：experience task、related task、trajectory context pool、oracle/free context、summary vs raw trajectory、accuracy/time/cost 三维评测。

读完后应该能回答：为什么“有记忆”不等于“会复用经验”；为什么 raw trajectory 通常太吵；为什么正确选择的短 summary 比长日志更有价值。

1 核心图景

普通 repo issue benchmark 把每个任务当作独立 episode：给 agent 一个 issue，让它修复，然后用 tests 判断结果。这个设置能评测单次修复能力，但不能回答一个更接近真实工程的问题：如果 agent 昨天刚解决过一个相似 bug，今天遇到相关 issue 时，它能不能复用昨天的经验？



SWE-ContextBench 的核心设计，是把过去任务的完整轨迹变成一个 context pool，再让 agent 在解决相关任务时检索、总结、选择和复用这些过去经验。这样评测不只看最终 resolved rate，还看 prior context 是否提高准确率、减少时间、降低 token 或成本。

Insight. SWE-ContextBench 不是在问“context window 能装多长”，而是在问“agent 能不能把过去一次解决问题的经验，转化成下一次相关问题的有效帮助”。

2 和 ContextBench 的区别

#15 的 ContextBench 关心的是单个 issue-resolution trajectory 内部：agent 有没有找到 gold context，是否过度检索，是否把读到的证据用在最终 patch 中。SWE-ContextBench 的问题更像跨任务记忆：过去某个 issue 的解决过程，是否能帮助未来相关 issue？

维度	ContextBench	SWE-ContextBench
核心问题	当前任务里有没有找到正确上下文	过去经验能否帮助相关新任务
主要对象	gold context 与 agent trajectory 的 overlap	experience task、related task、trajectory pool
失败类型	找不到、找太多、找到了但没用	总结差、检索错、复用盲目、成本上升
指标重点	recall、precision、F1、过程动态	accuracy variation、time、token/cost

两者不是重复，而是上下两层。#15 给出“找上下文”的显微镜；本讲给出“跨任务复用经验”的系统实验台。一个 agent 可能在当前任务里很会找文件，但不擅长从历史轨迹中抽取可迁移模式；也可能能检索历史案例，却把历史里的错误步骤照搬到新任务。

3 Experience Task 与 Related Task

SWE-ContextBench 把任务分成两类。Experience tasks 是过去已经运行过的任务，它们的 solution trajectories 被存进 context pool。Related tasks 是未来要解决的新任务，它们和某些 experience tasks 有真实的 issue / pull request 依赖或引用关系。

这个设计比随机相似度检索更强，因为 relatedness 来自真实软件开发记录。一个 PR 可能同时修复多个 issue；一个 issue 可能引用另一个 issue；一个 PR 可能引用另一个 PR；这些关系构成了“过去经验可能有用”的结构性理由。

对象	在 benchmark 中做什么	类比到真实工程
Experience task	生成可复用的完整轨迹	团队历史 debug 记录
Trajectory pool	存储工具调用、文件导航、推理过程	工程知识库 / agent memory
Related task	测试新 issue 是否受益于历史经验	相似 bug、同模块回归、相关 PR
Context setting	控制 agent 能看到什么过去信息	记忆系统、检索系统、summary 策略

这个区分很关键。Experience task 不是“训练样本”的简单别名，它在 benchmark 里扮演的是历史经验来源：agent 先在这些任务上产生轨迹，轨迹再被整理成未来可查的 memory。Related task 也不是随机抽出来的相似任务，而是由真实 issue / PR 关系支持的未来问题。也就是说，benchmark 先定义“过去经验确实可能相关”，再测试 agent 能不能把这种相关性转化成有效帮助。

如果没有这层结构，所谓 memory evaluation 很容易退化成普通语义检索：看 query 和历史文本是否相似。但软件工程里的经验复用往往不是表面文本相似，而是 bug 根因、模块边界、调用路径、测试失败模式和 patch pattern 的相似。一个 issue 可能没有使用同样的词，却共享同一个 parser corner case；另一个 issue 可能标题很像，但对应的是完全不同的 runtime behavior。

容易误读	更准确的读法	为什么影响评测
把历史任务当训练集	历史任务是可检索经验池	评测的是 inference-time memory，而不是参数更新
把 related 当文本相似	related 来自真实开发关系	能避免只测关键词匹配
把 trajectory 当文档	trajectory 是解决过程记录	其中既有证据，也有噪声和误判
把 context reuse 当 RAG	reuse 还包含适配和验证	找到相关案例不等于会修当前 bug

4 数据构造

论文报告的完整 SWE-ContextBench 包含 1,100 个 base / experience tasks 和 376 个 related tasks，总计 1,476 个任务，覆盖 51 个 GitHub repositories 和 9 种 programming languages。Lite 版本包含 300 个 base tasks 和 99 个 related tasks，用于更快、更便宜的评测。

来源	Experience tasks	Related tasks	Total
SWE-Bench Lite	300	99	399
SWE-Bench Multilingual	300	111	411
SWE-Bench Verified	500	166	666
SWE-ContextBench	1,100	376	1,476

Related tasks 来自六类真实关系：multi-issue resolution、pull-request-to-issue references、pull-request-only references、issue-to-issue references、issue-to-pull-request references、multi-references。论文还

用 recursive context expansion 找到第二阶依赖。这个构造方式的重点不是“数量更大”，而是把软件工程里真实存在的跨 issue 依赖显式化。

Relatedness 来源	数量	说明
multi-issue resolution	46	一个 PR 同时解决多个 issue，说明这些问题共享修复上下文。
pull-request-to-issue references	72	PR 引用其他 issue，可形成直接或同 PR 的相关任务。
pull-request-only references	34	PR 之间互相引用，常见于补丁链和后续修复。
issue-to-issue references	103	issue 之间互相引用，可能共享根因或模块。
issue-to-pull-request references	41	issue 指向已有 PR，暗示解决路径或实现位置相关。
multi-references + recursive	23 + 57	多重引用和二阶依赖，捕捉更长的工程关系链。

这张表比总任务数更重要，因为它解释了 benchmark 的信号来源。一个好的 cross-task context benchmark 不能只说“这些任务相似”，它必须说明相似性从哪里来。PR / issue 引用关系提供的是开发者留下的关系证据：这些任务在真实项目历史里曾经互相指向、共同解决、或者沿着同一个补丁链演化。

当然，这种构造也有风险。引用关系不一定意味着可迁移经验一定存在；有些引用只是讨论背景，有些同 PR 修复只是工程师顺手合并。因此 SWE-ContextBench 的合理读法不是“每个 related task 都应该被 memory 解决”，而是“这里存在足够真实的跨任务关联，可以用来评测 memory 系统能否发现和利用其中一部分”。

5 验证结构

每个 related task 仍然是可执行的软件工程任务。它有 base commit、problem statement、solution patch 和 test patch。评测时先运行原始测试得到 before，再应用 test patch 和 solution patch 得到 after，由此构造 FAIL_TO_PASS 和 PASS_TO_PASS。

测试集合	含义	读法
FAIL_TO_PASS	修复前失败、修复后通过的测试	是否解决目标 bug
PASS_TO_PASS	修复前后都通过的测试	是否避免 regression
Resolved	同时满足修复与回归约束	最终任务级成功

论文报告 376 个 related tasks 平均有 4.8 个 FAIL_TO_PASS tests 和 211.5 个 PASS_TO_PASS tests。这个比例很有代表性：bug fix 通常只有少数目标测试能证明修复，但需要大量回归测试证明没有破坏其他行为。

从系统设计角度看，FAIL_TO_PASS 和 PASS_TO_PASS 共同定义了“复用经验”的边界。一个 agent 如果照搬历史 patch，可能让目标测试通过，却破坏当前版本里已经正确的行为；这时 FAIL_TO_PASS 会奖励它，PASS_TO_PASS 会惩罚它。反过来，一个 agent 如果非常保守，几乎不改代码，PASS_TO_PASS 可能很高，但 FAIL_TO_PASS 不会通过。

Agent 行为	表面现象	指标上的暴露方式
照搬历史 diff	当前 bug 可能被修掉, 也可能引入新破坏	FAIL_TO_PASS 上升, PASS_TO_PASS 下降
只做保守小改	回归少, 但目标 bug 没解决	PASS_TO_PASS 高, FAIL_TO_PASS 低
读了 memory 但没用 正确迁移经验	成本上升, patch 与 baseline 类似 找到根因, 并适配当前版本	resolved 不变, time / cost 变差 resolved 上升, 回归不明显增加

因此, 本讲的 benchmark 不只是“带记忆能不能更准”, 而是“带记忆之后, agent 是否仍然能在可执行验证约束下做正确、有限、可回归的修改”。这和真实产品的关系很直接: memory 不能成为跳过测试的理由, 反而应该让测试证据更容易收集和解释。

6 Context Pool 是什么

Context pool 存的不是静态文档, 而是过去 experience tasks 的 solution trajectories。轨迹包含工具调用、文件导航、中间推理和最终总结。它更像“解决问题的过程记忆”, 而不是普通 RAG 文档库。

这也是 SWE-ContextBench 比一般 memory benchmark 更接近 coding agent 的地方。软件工程经验往往不是一段事实, 而是一条过程: 先看到失败测试, 再定位到模块, 再理解调用关系, 再尝试 patch, 再跑测试修正。如果把这条过程原样存下来, 它可能很有信息量, 也可能非常吵。

Insight. Trajectory memory 的难点是双重的: 它包含真实解决路径, 也包含试错、误判、无关文件和被后续修正的中间想法。复用历史轨迹必须先把“可迁移经验”和“过程噪声”分开。

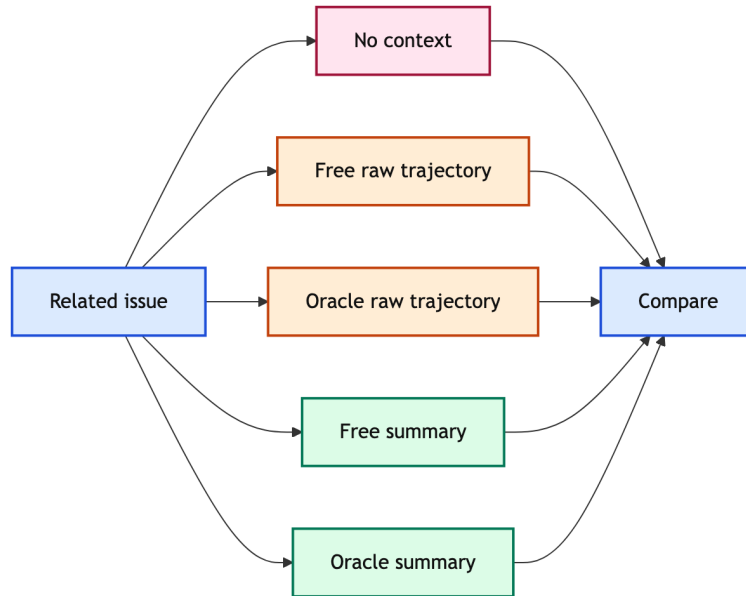
可以把 trajectory memory 分成四层对象。第一层是 observation: 测试输出、文件内容、命令结果。第二层是 action: 搜索、编辑、运行测试、回滚。第三层是 reasoning: agent 当时为什么认为某个文件相关。第四层是 outcome: 最终 patch 是否解决问题、哪些测试通过、哪些尝试被放弃。对 memory 系统来说, 这四层的价值不同。

Trajectory 层	可复用价值	主要噪声
Observation	失败测试、错误栈、文件证据可迁移	旧版本路径、偶然日志、环境差异
Action	显示有效定位路径和验证路径	无效搜索、重复命令、探索分支
Reasoning	暴露根因假设和模块解释	中间误判、过度自信、被修正的假设
Outcome	最终 patch rationale 和验证结论	不完整验证、局部成功、残余 bug

这也解释了为什么 raw trajectory 平均 25k tokens 不一定有优势。完整轨迹保留了很多证据, 但这些证据并没有自动分层。一个 agent 在新任务里读到旧轨迹时, 必须知道哪些 observation 仍然相关, 哪些 action 只是历史试错, 哪些 reasoning 已被后续结果推翻, 哪些 outcome 才是可迁移经验。

7 五种 Context Setting

论文在 Lite 设置中比较了五种 prior context 暴露方式。它们构成一个很好的实验矩阵: 没有上下文、自由检索原始轨迹、oracle 给原始轨迹、自由检索 summary、oracle 给 summary。



Setting	Agent 能看到什么	主要隔离的问题
No-Context	只有当前 issue	baseline
Free Context	自己从原始轨迹池找	自主检索 raw memory 是否有用
Oracle Context	直接给最相关原始轨迹	去掉 retrieval error 后 raw context 是否有用
Free Summary	自己从 summary 池找	自主检索 compact memory 是否有用
Oracle Summary	直接给最相关 summary	高质量短经验的上限

这个矩阵最重要的地方，是把 representation 和 selection 分开。Raw trajectory vs summary 是表示问题；free vs oracle 是选择问题。一个系统失败时，要问它失败在“记忆太长太吵”，还是失败在“没找对记忆”。

对比	隔离出的瓶颈	读结果时间什么
No vs Oracle Summary	高质量经验本身是否有用	如果这里没提升，说明相关经验也许不可迁移。
Oracle Context vs Oracle Summary	原始轨迹和短总结哪个更可用	如果 summary 更强，说明压缩质量比完整性重要。
Free Summary vs Oracle Summary	自主检索是否找对 summary	差距大时，优先改 retriever / index。
Free Context vs Oracle Context	自主检索 raw trajectory 是否可控	差距大时，说明长轨迹选择和阅读都很难。
Free Context vs No Context	自主 memory 是否值得开启	如果成本上升且准确率不变，应默认关闭或自适应开启。

这张诊断表在工程上很有用。很多团队看到 memory 系统表现不好，会直接换 embedding 或扩大 top-k。但 SWE-ContextBench 的 setting 告诉我们：先用 oracle summary 测上限，再测 free summary 的检索差距；先确定表示有价值，再优化选择机制。否则，团队可能在一个本来就没有可迁移经验的任务集合上反复优化 retriever。

8 Raw Trajectory vs Summary

论文报告 summary 文件平均 217.1 tokens，而完整 context trajectories 平均 25,633.7 tokens。这个对比说明，agent memory 的核心不是把历史全部塞回 prompt，而是把历史变成可检索、可读、可迁移的经验单元。

表示形式	优点	风险
Raw trajectory	保留完整证据、工具调用和试错过程	太长、噪声大、容易复用错误步骤
Final summary	短、便宜、可排序、容易读	可能丢掉关键细节或过度概括
Structured memory	可以按 repo、任务类型、依赖关系组织	需要额外 schema 和维护成本

这和 #13 的 context engineering 直接相连。长 context 本身不是知识，只有被压缩成合适粒度、带有元数据、能被检索和验证的经验，才可能成为 agent 的可复用记忆。

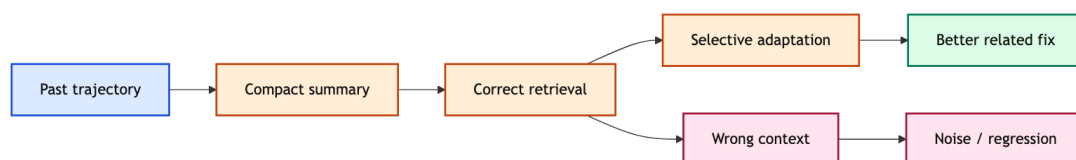
好的 summary 不是普通摘要。普通摘要会说“修复了某个 bug，修改了某个文件”；可复用经验 summary 需要说明：失败信号是什么，根因是什么，为什么这个文件是关键，最终 patch 改变了什么语义，哪些测试证明修复，哪些条件下不要套用。换句话说，它应该是一个小型工程 case card，而不是聊天记录压缩。

Summary 字段	应该记录	不够好的写法
Symptom	失败测试、错误栈、用户可见行为	“测试失败了”
Root cause	数据结构、边界条件、调用约定的具体问题	“逻辑有 bug”
Evidence	哪些文件、函数、日志支持判断	“检查了相关代码”
Patch pattern	修改的语义，而不是只列文件名	“更新了一些实现”
Transfer guard	当前经验适用和不适用的条件	“类似问题可以参考”

这也是 Oracle Summary Learning 能提升的一个可能原因：当 summary 足够短而且直接指向根因时，它降低了 agent 在新任务里重新探索的成本；当 summary 过于泛化或选错时，它反而会提供错误先验。未来的 agent memory 不应该只追求压缩率，而应该追求“压缩后仍保留可迁移判断”。

9 复用链路

把过去经验用于新 issue，至少要经过四步：总结过去轨迹、检索正确案例、抽取可迁移模式、选择性适配当前任务。任何一步失败，context learning 都可能变成负担。



这条链路解释了为什么“有记忆”不等于“会学习”。一个 agent 可以检索到错误案例，或者检索到正确案例但照搬错误中间步骤，或者拿到正确 summary 但没有识别当前任务的差异。有效复用需要 agent 既能相信过去经验，又能质疑过去经验。

诊断问题:

如果 prior context 没有提升表现, 依次问: summary 是否有用? retriever 是否找对? agent 是否正确解释? 当前 issue 是否真的需要复用历史经验?

10 论文结果怎么读

在五种 context setting 中, 论文报告 Oracle Summary Learning 在 99 个 Lite related tasks 上把 resolved rate 从 No-Context 的 26.26% 提高到 34.34%。Oracle Context Learning 只到 27.27%, Free Summary Learning 反而是 22.22%。这说明正确选择的短 summary 很有价值, 但错误或自由选择的 summary 可能误导 agent。

Setting	Resolved	Time	Cost
No-Context	26.26%	6.37 min	\$0.79
Free Context	26.26%	6.78 min	\$0.98
Oracle Context	27.27%	5.95 min	\$0.77
Free Summary	22.22%	6.28 min	\$0.91
Oracle Summary	34.34%	6.66 min	\$0.85

这里不能只读 resolved rate。Free Context 没有提高 resolved rate, 却提高了成本; Oracle Context 最快且成本最低, 但 resolved 提升很小; Oracle Summary resolved 最高, 但 cost 不最低。这说明 context learning 是 accuracy、runtime、token/cost 的三目标 tradeoff。

还有一个容易忽略的点: Oracle Summary 的回归保持率低于 No-Context, 但目标修复率更高。这意味着它更积极地修复目标 bug, 但可能带来更多回归压力。这个结果不是坏事, 也不是好事; 它提醒我们, memory 可能改变 agent 的风险偏好。读到相关历史经验后, agent 可能更敢改动关键代码, 因此目标修复率上升, 但回归风险也要用 tests 盯住。

结果现象	可能原因	工程动作
Oracle Summary 最准	正确短经验降低定位成本	优先建设 summary quality 和 case card
Free Summary 低于 base-line	选错 summary 或过度相信 summary	增加 confidence、source、适用条件
Free Context 成本最高	raw trajectory 太长, 选择也不稳定	不默认开放全文轨迹检索
Oracle Context 最快	给定正确轨迹后减少探索尾部	用 oracle 实验估计 memory 上限
Memory framework 分化	ingestion / retrieval / ranking 机制不同	单独记录 match、tokens、resolved

所以, SWE-ContextBench 最有价值的地方不是给出一个最终排行榜, 而是把 memory 系统拆成可调试的组件。resolved rate 只能告诉你系统是否有效; setting 对比和 retrieval table 才能告诉你系统为什么有效或为什么无效。

11 为什么 Free Retrieval 会伤害

Free Context Learning 看似最接近真实产品: 给 agent 一个记忆池, 让它自己决定查什么。但论文结果显示, free retrieval 不一定带来收益。Table 5 里 Free Context Learning 的 top-1 retrieved context 平均达到 136k tokens, match rate 只有 18.18%。这是一种典型的“读很多但没读对”。

Free Summary Learning 的 top-1 context 平均约 373 tokens, match rate 提高到 34.34%, 但 resolved rate 仍然低于 baseline。这说明短 summary 解决了长度问题, 却没有自动解决选择和利用问题。一个错的短 summary 可能比没有 memory 更危险, 因为它看起来像“经验”, 会给 agent 错误的先验。

Insight. Memory 的危险不只是噪声，而是带有权威感的噪声。过去轨迹一旦以 summary 形式出现，agent 可能过度相信它；因此 memory 系统必须同时优化 relevance、trust 和 adaptation。

Free retrieval 还会引入一个预算分配问题。Agent 在读历史 context 上花掉的 token 和时间，可能挤占当前任务的探索预算。如果检索结果正确，它能减少重复探索；如果检索结果错误，它会让 agent 在错误方向上探索更久。长上下文尤其明显：读 136k tokens 的旧轨迹，不只是贵，它还会把当前 issue 的局部证据淹没在历史细节里。

伤害机制	具体表现	缓解办法
Selection error	找到不相关历史任务	用 repo / module / relation metadata 先过滤
Representation noise	历史轨迹太长、试错太多	先总结为 case card，再按需展开 raw log
Authority bias	agent 过度相信旧 summary	summary 里加入 confidence 和 transfer guard
Budget crowding	历史 context 挤占当前探索	对 memory 使用设置 token 和时间预算
Version drift	旧 patch 不适合当前代码版本	要求重新验证调用关系和 tests

这张表也可以作为产品开关策略：不是每个任务都要启用 memory。对于小改动、明确错误栈、定位非常直接的 issue，No-Context 可能更快；对于跨模块、回归、重复 bug、框架升级、相似失败测试，memory 才更可能带来收益。

12 Memory Framework 结果

论文还比较了 Mem0、OpenViking、LangMem、Supermemory 等 memory retrieval frameworks。结果显示，retrieval matching 和最终 resolved 不是同一个东西。LangMem 在 Table 5 中 overall match rate 最高，达到 73.34%；Supermemory 的 overall match rate 为 59.60%，但在 Table 4 中 resolved rate 最高，为 30.30%。

Method	Overall match	Resolved	Cost
Mem0	39.39%	24.24%	\$0.62
OpenViking	51.52%	29.20%	\$0.53
LangMem	73.34%	-	-
Supermemory	59.60%	30.30%	\$0.58

这组结果应该谨慎读。Match rate 衡量是否找到了正确历史案例；resolved rate 衡量最终是否修复当前任务。中间还隔着理解、过滤、适配、编辑和验证。因此更高 retrieval match 是必要能力，但不是充分条件。

框架差异	在 benchmark 中体现为	对 agent 产品的启发
Ingestion 方式	存完整对话、摘要、metadata 或资源索引	memory schema 会决定后续能否按工程关系检索
Retrieval 粒度	返回几十 tokens、几百 tokens 或长轨迹	粒度太小可能缺证据，太大可能淹没任务
Ranking 策略	top-1 / top-k match rate 差异	排名质量需要单独评测，不应只看最终 resolved
Utilization 接口	agent 如何读取和引用 memory	memory 需要变成可操作证据，而不是旁路提示

一个重要结论是：memory framework 不是外接插件那么简单。它决定 agent 看到的是“一个相关历史案例的提炼经验”，还是“一堆不知真假的旧对话”。当 action space 是代码修改时，memory 的输出最好接近 evidence card：包含来源、相关模块、历史根因、适用条件和验证方式。否则，memory 只能增加 prompt 字数，不能稳定增加工程能力。

13 从 Benchmark 到产品 Memory

把 SWE-ContextBench 的思想放进产品，需要一个更明确的 memory lifecycle。第一步是 capture：每次 agent 完成任务后，记录 issue、patch、测试结果、失败尝试和最终解释。第二步是 distill：把轨迹压缩成结构化 case card。第三步是 index：按 repository、模块、语言、错误类型、测试信号和依赖关系建索引。第四步是 retrieve：根据当前任务决定是否查 memory、查几条、用什么粒度。第五步是 verify：把复用经验转化成当前任务里的可执行测试和 patch 证据。

Lifecycle	产物	质量门槛
Capture	原始 trajectory、patch、测试日志	轨迹完整，能回放关键证据
Distill	case card / summary	包含 root cause、patch rationale、transfer guard
Index	repo / module / issue relation metadata	支持先过滤，再语义检索
Retrieve	top-k memory candidates	记录 match confidence 和 token cost
Verify	当前任务测试与 patch evidence	不能只引用历史成功，必须当前验证

这条 lifecycle 也能解释为什么很多“长期记忆”产品体验不稳定：它们只做 capture 和 retrieve，中间缺少 distill、index 和 verify。对 coding agent 来说，memory 必须能被审计。用户或 reviewer 应该知道 agent 为什么引用某个历史任务、它从中学到了什么、哪些地方没有直接套用、最终用什么当前证据证明 patch 有效。

14 Agent 产品里的设计启发

SWE-ContextBench 给 coding-agent 产品提供了四条直接启发。第一，历史轨迹要总结成经验，而不是原样塞回上下文。第二，memory 要有结构：repository、issue 类型、依赖关系、失败模式、patch 模式都可以成为索引键。第三，retrieval 需要自适应：简单任务不一定需要查历史，复杂任务可能需要多个相关案例。第四，agent 必须学会批判性复用，而不是复制过去 patch。

设计层	好做法	坏味道
Summarization	记录根因、证据、最终 patch rationale	只存聊天流水账
Indexing	按 repo、模块、bug 类型、依赖关系组织	所有记忆混在一个向量池
Retrieval	根据任务复杂度调 top-k 和粒度	固定取若干条 memory
Utilization	明确哪些经验可迁移，哪些要忽略	盲目照搬历史 diff

15 怎样做内部实验

如果团队想把 SWE-ContextBench 思想迁移到自己的 agent，不必马上构造完整 benchmark。可以先从 related-issue pairs 开始：选择同一 repository 中引用关系明确、模块相近、历史修复记录完整的一组 issue，把过去的轨迹整理成 memory pool，再比较 No-Context、Free Summary、Oracle Summary 三个 setting。

最小实验报告应该同时记录四类指标：resolved rate、平均 runtime、平均 token/cost、retrieval match。还要保存失败样例，说明 prior context 是没找到、找错、找对但没用，还是用错。这样团队能知道下一步该改 summarizer、retriever、memory schema，还是 patch generator。

实验项	推荐做法	目的
Related pairs	从真实 issue / PR 引用关系构造	确保历史经验确实可能有用
Oracle summary	人工指定最相关历史 summary	测上限，隔离 retrieval error
Free summary	让 agent 自己检索 summary	测真实 memory system
Failure audit	对失败轨迹做归因	找到系统瓶颈

实验完成后，不要只写一个平均分。更有用的是按失败类型归档。至少区分五类：没有检索到相关经验、检索到了错误经验、检索到了正确经验但没有用、用了经验但没有适配当前版本、用了经验且目标修复成功但引入回归。这个失败归因会直接决定下一步改哪里。

失败类型	观察信号	下一步优化
No retrieval	top-k 没有相关历史任务	改 index、metadata、query rewrite
Wrong retrieval	summary 看似相关但 gold relation 不匹配	改 ranking、加入 relation filter
Unused retrieval	prompt 中有正确 summary，但 patch 无关	改 agent instruction 和 evidence use
Bad adaptation	照搬旧 patch，当前测试失败	加 transfer guard 和 version check
Regression	目标测试通过，回归测试失败	强化 PASS_TO_PASS 和 review policy

最小实验结论模板：

本轮 memory 是否提升 resolved？是否增加成本？retrieval match 是否提升？失败主要发生在 selection、representation、utilization 还是 verification？下一轮只改一个组件，并保留 No-Context / Oracle Summary 两个参照。

16 局限与开放问题

论文也指出 SWE-ContextBench 有限制。Related tasks 虽然经过人工识别和结构验证，但没有对每个任务做全面的人类可解性和测试质量审查。Context pool 也没有严格过滤，只是反映真实工程轨迹可能包含试错、部分正确、未解决或带有残余 bug 的情况。

这些限制其实也揭示了真实产品问题。企业里的历史记录并不总是干净的：旧 issue 可能修错过，PR 讨论可能包含过期判断，测试可能不足，经验可能只适用于旧版本。因此 agent memory 的难点不是存储，而是判断一段历史经验在当前上下文中是否仍然可信。

还有三个开放问题值得后续继续追踪。第一，summary 是人工规则、LLM 自动生成，还是由成功轨迹训练出来的？不同 summary 生成方式会改变 benchmark 的上限。第二，memory 应该在任务开始前一次性检索，还是随着 agent 的探索动态更新？真实调试常常是先看到失败测试，再知道该查

哪类历史。第三，memory 是否会造成 benchmark leakage? 如果历史轨迹包含过于接近的 patch 或 oracle 信息，agent 的提升可能来自复读答案，而不是学会迁移。

开放问题	为什么难	后续 notes 里的连接点
Summary 生成	好 summary 需要区分根因、证据和适用条件	skills、procedural memory
Dynamic retrieval	当前探索会改变“相关经验”的定义	long-running harness
Leakage control	历史经验可能太接近答案	benchmark leakage、reward hacking
Memory governance	旧经验可能过期、错误或有权限问题	sandbox、permission、enterprise workflow

17 最终 Takeaway

SWE-ContextBench 的核心意义，是把 coding agent 的“经验复用能力”从单次 issue repair 中拆出来。它让我们看到：过去轨迹必须被总结、检索、筛选和适配，才能变成对新任务有帮助的 context。

对后续 notes 来说，本讲提供了一个重要结论：long-horizon agent 的 memory 不是越多越好。真正重要的是，记忆是否 compact、相关、结构化、可验证，并且 agent 是否知道什么时候该用、什么时候该忽略。

关键词	在本讲中的含义
Experience task	用来生成过去轨迹和经验池的任务。
Related task	与过去任务有真实 issue / PR 关系的新任务。
Context learning	从过去任务经验中提高新任务准确率、时间或成本表现。
Oracle summary	人工指定最相关短经验，用来测高质量 memory 的上限。
Free retrieval	agent 自主从 memory pool 中找经验，最接近真实产品。

参考资料

- [1] [SWE Context Bench](#). benchmark construction, context settings, memory retrieval comparisons, and limitations
- [2] [SWE-bench](#). background lineage for repository-level issue-repair tasks